

TokenBuySellApp

A Django application for purchasing ERC-20 tokens on the Ethereum blockchain

Technical Documentation · Sara Rostami Pour · University of Tehran

1. Overview

TokenBuySellApp lets a registered user convert an off-chain credit balance into **SaraToken (SAR)**, a custom ERC-20 token deployed on the Ethereum Sepolia test network. The user supplies an Ethereum wallet address at registration; when they buy tokens, the backend signs and broadcasts a real on-chain transfer to that address.

The application was originally an undergraduate final project. It has since been refactored: credentials were removed from the source code, the blockchain layer was extracted into a dedicated service module, the purchase flow was made safe against partial failure, and a test suite was added.

2. The two-ledger model

The central design idea is that the system tracks *two separate balances* for each user, and a purchase is the operation that moves value from one to the other.

Ledger	Where it lives	What it represents
Credit balance	Django database Account.balance	Off-chain currency the user has charged into their account. Fully under the application's control.
Token balance	Ethereum blockchain FirstToken contract	Real ERC-20 tokens owned by the user's wallet. The application cannot alter this directly — only the blockchain can.

This distinction matters because the two ledgers have very different failure modes. A database write can be rolled back; a broadcast Ethereum transaction cannot. The purchase flow is designed around that asymmetry.

3. What ERC-20 actually is

ERC-20 is not a piece of software — it is an *interface*. It specifies six functions and two events that a contract must expose in order for wallets, exchanges and block explorers to recognise it as a token. Any contract implementing this interface is automatically compatible with the entire Ethereum tooling ecosystem.

Function	Purpose
<code>totalSupply()</code>	How many token units exist in total
<code>balanceOf(address)</code>	How many units an address owns
<code>transfer(to, amount)</code>	Move tokens from the caller to another address

<code>approve(spender, amount)</code>	Authorise another address to spend on the caller's behalf
<code>allowance(owner, spender)</code>	How much a spender is still authorised to move
<code>transferFrom(from, to, amount)</code>	An approved spender moves tokens on the owner's behalf

The two events, `Transfer` and `Approval`, are what block explorers such as Etherscan index in order to display a token's transaction history. Without emitting them, a token would move value but appear invisible to the ecosystem.

4. The smart contract

The contract lives in `contracts/TokenFirst.sol` and is written for Solidity 0.8.14.

Property	Value
Token name	SaraToken
Symbol	SAR
Decimals	10
Total supply	10^{12} base units = 100 whole tokens
Network	Sepolia testnet

4.1 Decimals and base units

The Ethereum Virtual Machine has no floating-point arithmetic. Token amounts are therefore stored as integers, and the `decimals` field tells clients where to place the decimal point for display. With `decimals = 10`:

```
1 whole token = 10^10 = 10,000,000,000 base units
total supply = 10^12 base units = 100 whole tokens
```

Every conversion between the two representations happens exactly once, in `TokenService.to_base_units()`. Keeping the conversion in a single place prevents the classic bug of accidentally multiplying or dividing twice.

4.2 The treasury pattern

The constructor mints the entire supply to `msg.sender` — the wallet that deploys the contract. That wallet becomes the **treasury**. It is the only address holding tokens initially, and every purchase is a transfer out of it.

```
constructor() {
    _totalSupply = 1_000_000_000_000;
    _balances[msg.sender] = _totalSupply;
    emit Transfer(address(0), msg.sender, _totalSupply);
}
```

The `Transfer` event from the zero address is the ERC-20 convention for signalling a mint. Emitting it lets Etherscan display the initial supply correctly.

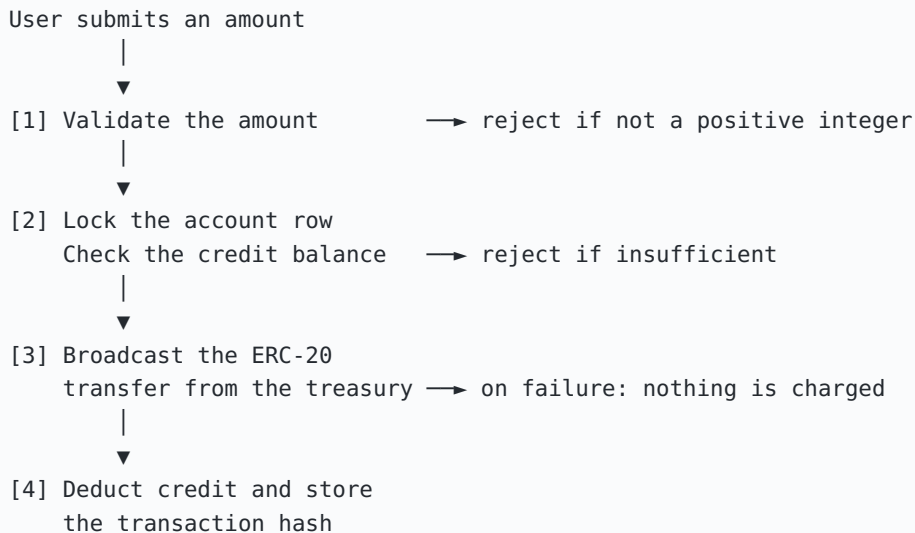
4.3 Why SafeMath was removed

The original contract imported a `SafeMath` library to guard against integer overflow. Before Solidity 0.8, arithmetic wrapped silently: subtracting 1 from an unsigned zero produced the maximum `uint256` value, an exploitable bug.

Since Solidity 0.8.0, the compiler inserts overflow and underflow checks automatically and reverts the transaction on violation. `SafeMath` is therefore redundant, and removing it makes the contract cheaper to execute and easier to read.

5. The purchase flow

This is the most safety-critical path in the application, because it spans two systems that cannot participate in a shared transaction. A Django database transaction can be rolled back; an Ethereum transaction, once broadcast and mined, is permanent.



5.1 Why the on-chain transfer comes first

The ordering of steps 3 and 4 is deliberate. Consider the alternative — deducting the credit first, then transferring:

- If the transfer then fails or reverts, the user has been charged for tokens they never received.
- Refunding requires detecting the failure, which is unreliable if the process crashes between the two operations.

With the current ordering the worst case is inverted: if the process dies after the transfer but before the deduction, the user receives tokens they were not charged for. That is a loss for the operator, not for the user — the correct direction for the error to fall in a system handling other people's money.

5.2 Preventing double-spending

Two concurrent requests could each read the same credit balance, each conclude that the user can afford the purchase, and each proceed. The account would be debited once but tokens transferred twice.

```
with db_transaction.atomic():
    account = Account.objects.select_for_update().get(pk=request.user.pk)
    if account.balance < cost:
        ...reject...
```

`select_for_update()` issues a `SELECT ... FOR UPDATE`, taking a row-level lock in the database. The second request blocks until the first commits, then re-reads the updated balance and correctly finds it insufficient.

6. Signing and broadcasting a transaction

A transaction that changes blockchain state must be cryptographically signed by the private key of the sending account. The application never sends the private key over the network; it signs locally and transmits only the signed payload.

```
tx = contract.functions.transfer(recipient, value).build_transaction({
    "from":      treasury_address,
    "nonce":     w3.eth.get_transaction_count(treasury_address),
    "gas":       200_000,
    "gasPrice":  w3.eth.gas_price,
})

signed    = w3.eth.account.sign_transaction(tx, private_key=TREASURY_PRIVATE_KEY)
tx_hash   = w3.eth.send_raw_transaction(signed.raw_transaction)
receipt   = w3.eth.wait_for_transaction_receipt(tx_hash, timeout=180)

if receipt.status != 1:
    raise BlockchainError("The blockchain transaction was reverted.")
```

Field	Meaning
nonce	A per-account counter. It fixes the order of an account's transactions and makes replay attacks impossible. Reusing a nonce causes the network to reject the transaction.
gas	The maximum computational work the transaction may consume. An ERC-20 transfer typically costs about 50,000 gas; 200,000 is a safe ceiling.
gasPrice	The price paid per unit of gas. Fetched live from the node rather than hardcoded, so the transaction is not underpriced when the network is busy.
receipt.status	<code>1</code> means the transaction executed successfully; <code>0</code> means it was mined but <i>reverted</i> . Checking this is essential — a mined transaction is not necessarily a successful one.

A mined transaction can still have failed. If `receipt.status` is `0`, the contract reverted — for example because the treasury ran out of tokens. The gas is still consumed, but no tokens moved. Code that assumes "mined" means "succeeded" will credit users for transfers that never happened.

7. Reading balances

Two different mechanisms read the on-chain balance, and they serve different purposes.

7.1 Direct contract call

`TokenService.balance_of()` calls `balanceOf()` on the contract through the RPC node. This is a *read*, not a transaction: it costs no gas, is not mined, and returns the authoritative current value.

7.2 Etherscan API

The template context processor queries the Etherscan API instead. This avoids an RPC round trip on every page render and gives the same result, at the cost of depending on a third-party service.

Because that service can be slow or unavailable, the context processor is written to never raise. A timeout, an HTTP error, or a malformed response all degrade to a displayed balance of zero rather than a 500 error — a context processor runs on *every* request, so an exception there would take down the entire site.

8. Architecture

Blockchain code is isolated in `myapp/services.py`. Views do not import `web3` at all.

Module	Responsibility
<code>myapp/views.py</code>	HTTP concerns only: parse input, enforce authentication, choose a response.
<code>myapp/services.py</code>	<code>TokenService</code> : connect to the node, convert units, sign and broadcast transfers, read balances. Raises <code>BlockchainError</code> on any failure.
<code>myapp/models.py</code>	Persistence: accounts, orders, invoices, transaction history.
<code>contracts/</code>	Solidity source and the compiled ABI.
<code>mysite/settings.py</code>	Configuration, read entirely from environment variables.

This separation has a concrete payoff: the purchase flow can be tested without a network connection, a funded wallet, or a deployed contract. The test suite replaces `TokenService` with a mock and asserts the invariants that matter — that a failed transfer never debits the user, that an insufficient balance is rejected before any transfer is attempted, and that concurrent purchases cannot double-spend.

9. Security

9.1 Credential handling

Every secret — the Django secret key, the RPC provider URL, the treasury address, the treasury private key, and the Etherscan API key — is read from environment variables loaded from a `.env` file that is excluded from version control. A committed `.env.example` documents which variables are required without revealing any value.

The treasury private key controls the entire token supply. Anyone holding it can transfer every token out of the treasury. It must never appear in source code, in a commit, in a screenshot, or in a log line. Use a wallet created solely for this project, on a testnet, holding nothing of value.

9.2 Input validation

The requested amount is parsed as an integer and rejected unless it is strictly positive. Wallet addresses are converted with `Web3.to_checksum_address()`, which raises on a malformed address rather than silently sending tokens into an unrecoverable one.

9.3 Failure isolation

All Web3 exceptions are caught in the service layer and re-raised as `BlockchainError`. Views handle a single exception type and present a friendly message; raw exception text, which may leak provider URLs or internal state, is written to the log rather than to the page.

10. Limitations

- The treasury signs every transfer, so the application is custodial: users trust the operator to send the tokens they paid for.
- There is no on-chain sell path. Selling tokens back would require the user to `approve()` the treasury and the backend to call `transferFrom()`.
- The gas limit is fixed rather than estimated per call.
- Deployed only on Sepolia. Nothing in the project has been audited, and it should not be used with assets of real value.

11. Running the project

```
git clone https://github.com/rossaYY/TokenBuySellApp.git
cd TokenBuySellApp

python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env             # then edit .env with your own values

python manage.py migrate
python manage.py runserver
```

Run the test suite — it mocks the blockchain and needs no network access:

```
python manage.py test
```

Stack: Python · Django 4.2 · web3.py · Solidity 0.8 · SQLite · Etherscan API
Sara Rostami Pour · sara.rostamipoor@ut.ac.ir · github.com/rossaYY